SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Mohan Kumar J	**Reg. No.:**	192424060
Section / Batch:	AI & DS	**Date:**	23-07-2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
	GRAND TOTAL:	**100 Marks**	

SECTION A – DEBUGGING & OPTIMISATION TASK

Question 1: Debugging Maximum Element Search Code (Empty List and Initialization Risk)

The following code is intended to find the maximum element, but it fails for empty arrays and uses an unsafe initialization pattern that may lead to errors. Carefully analyze the assumptions made by the code and explain why robustness is affected. Apply systematic debugging to handle edge cases correctly while preserving proper maximum search behavior. Optimize the implementation for clarity and defensive programming. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def find_max(arr):
    max_val = arr[0]    # ISSUE
    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val
```

Question 2: Debugging Binary Search Code (Incorrect Midpoint Formula with Bias)

The following Binary Search implementation computes the midpoint using a biased formula that can skip important candidates and distort the search interval for some inputs. Carefully inspect the midpoint calculation and explain why the search space is not divided correctly. Debug the algorithm to use a proper midpoint computation and consistent boundary updates. Optimize the implementation for readability and accurate handling of all edge cases. Analyze the corrected algorithm in best, average, and worst cases, and rewrite the final code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high + 1) // 2 + 1    # ERROR

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1
```

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Mohan Kumar J

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Debugging Maximum Element Search Code (Empty List and Initialization Risk)

Problem Identification

The given program attempts to find the maximum element in an array but directly initializes max_val with arr[0]. This causes an Index Error when the input list is empty. The algorithm also assumes that every input contains at least one element and does not validate the input before processing. These issues reduce robustness and reliability.

Given Code

```
def find_max(arr):
    max_val = arr[0]
    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val
```

Errors Identified

1. Accessing arr[0] on an empty list raises an IndexError.
2. Unsafe initialization assumes a non-empty list.
3. No input validation is performed before processing.

Systematic Debugging Process:

Step1: Check whether the input list is empty.

Step 2: Raise a meaningful ValueError if the list is empty.

Step 3: Initialize max_val only after validation.

Step 4: Traverse the remaining elements and update the maximum value.

Step 5: Return the maximum element.

Corrected Python Code

```
def find_max(arr):
    """Returns the maximum element in a list."""
    if not arr:
        raise ValueError("Array is empty. Maximum element
cannot be found.")
    max_val = arr[0]
    for value in arr[1:]:
        if value > max_val:
            max_val = value
    return max_val
```

Time Complexity Analysis:

Operation	Complexity
Empty list check	O(1)
Initialization	O(1)
Traversing list	O(n)
Total Time Complexity	**O(n)**
Space Complexity	**O(1)**

Explanation:

- Every element is visited only once.
- No extra array or auxiliary data structure is used.
- Therefore,

  Time Complexity = O(n)

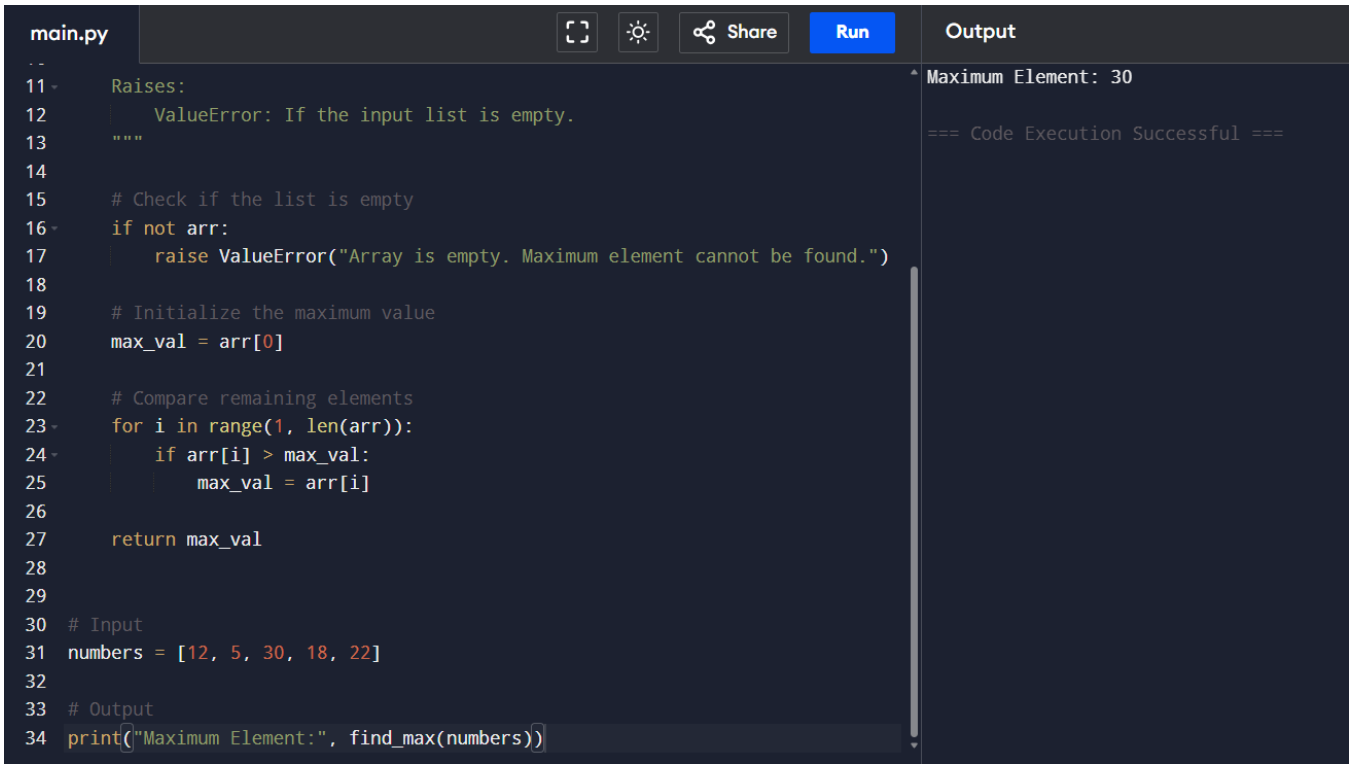
  Space Complexity = O(1)

Analysis and Justification

The corrected algorithm is more reliable because it validates the input before accessing the first element. It preserves the optimal linear search behavior while improving robustness through defensive programming. No additional memory is required.

Conclusion

The corrected implementation safely handles empty lists, prevents runtime errors, and maintains optimal O(n) time complexity with O(1) auxiliary space.



Question 2: Debugging Binary Search Code (Incorrect Midpoint Formula with Bias)

Problem Identification

The given binary search implementation calculates the midpoint using an incorrect formula that introduces a right-side bias. This can skip candidate elements, produce incorrect search intervals, and even result in an IndexError.

Given Code

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high + 1)//2 + 1
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Errors Identified

1. Incorrect midpoint calculation.
2. Search interval is divided unevenly.
3. Candidate elements may be skipped.
4. Risk of index out of range.
5. Missing documentation.

Systematic Debugging Process:

Step 1: Replace the midpoint calculation with low + (high - low) // 2.

Step 2: Divide the search interval evenly.

Step 3: Update low and high correctly.

Step 4: Continue until low > high.

Step 5: Return -1 if the key is not found.

Corrected Python Code

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = low + (high - low)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Time Complexity Analysis:

Case	Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)
Space Complexity	O(1)

Explanation

- In every iteration, Binary Search reduces the search space by half.
- Therefore,

  n

  n/2

  n/4

  n/8

  ...

  1

The number of iterations is

$$\log_2 n$$

Hence,

Best Case = O(1)

Average Case = O(log n)

Worst Case = O(log n)

Space Complexity = O(1)

Analysis and Justification

The corrected algorithm computes the midpoint accurately and ensures that every candidate element is considered. It avoids bias, prevents boundary errors, and follows the standard binary search implementation while retaining logarithmic efficiency.

Conclusion

The corrected binary search is robust, accurate, and efficient. It correctly handles all search cases with O(log n) time complexity and O(1) space complexity.